

Flow Decode của HQC với mã nối tầng Reed–Muller và Reed–Solomon

1 Mục tiêu và vị trí của decode trong HQC

Trong HQC, ciphertext của PKE có dạng

$$c = (u, v),$$

trong đó

$$u = r_1 + r_2 h \in R = \mathbb{F}_2[X]/(X^n - 1),$$

và

$$v = C.\text{Encode}(m) + \text{Truncate}(sr_2 + e).$$

Khóa bí mật chứa seed để sinh lại vector bí mật y . Khi decrypt, HQC tính

$$w = v - \text{Truncate}(uy).$$

Vì làm việc trên $\mathbb{F}_2$, phép trừ chính là XOR:

$$w = v + \text{Truncate}(uy).$$

Sau đó w được đưa vào decoder của mã nối tầng:

$$m' = C.\text{Decode}(w).$$

Trong code tham chiếu, flow này tương ứng với:

```
// hqc.c
vect_mul(tmp1, y, c_pke->u);
vect_truncate(tmp1);
vect_add(tmp2, c_pke->v, tmp1, VEC_N1N2_SIZE_64);
code_decode(m, tmp2);
```

Ví dụ nhỏ

Giả sử bỏ qua kích thước thật của HQC và xét vector bit:

$$v = 10110100, \quad \text{Truncate}(uy) = 00100110.$$

Khi đó đầu vào cho decoder là

$$w = v + \text{Truncate}(uy) = 10110100 \oplus 00100110 = 10010010.$$

Trong HQC thật, w không chỉ dài vài bit, mà có độ dài

$$n_1 n_2$$

bit, chính là độ dài của codeword nối tầng RMRS.

2 Mã nối tầng C : RS bên ngoài, RM bên trong

HQC dùng mã nối tầng:

$$C = C_{\text{RM}} \circ C_{\text{RS}}.$$

Encode đi theo thứ tự:

$$m \xrightarrow{\text{RS.encode}} c_{\text{RS}} \xrightarrow{\text{RM.encode}} c_{\text{RMRS}}.$$

Decode đi ngược lại:

$$w \xrightarrow{\text{RM.decode}} \tilde{c}_{\text{RS}} \xrightarrow{\text{RS.decode}} m'.$$

Trong code:

```
// code.c
void code_decode(uint64_t *m, const uint64_t *em) {
    uint64_t tmp[VEC_N1_SIZE_64] = {0};

    reed_muller_decode(tmp, em);
    reed_solomon_decode(m, tmp);
}
```

2.1 Tham số chính

Variant	n	n_1	n_2	k byte	δ	RM multiplicity
HQC-128	17669	46	384	16	15	$384/128 = 3$
HQC-192	35851	56	640	24	16	$640/128 = 5$
HQC-256	57637	90	640	32	29	$640/128 = 5$

Table 1: Các tham số lấy từ `parameters.h`.

Ý nghĩa:

- n : độ dài vector/polynomial trong ring HQC.
- n_1 : số symbol byte của codeword Reed–Solomon sau encode.
- n_2 : số bit mà mỗi byte RS được encode thành bởi duplicated RM.
- k : số byte thông điệp.
- δ : khả năng sửa lỗi của RS decoder.
- RM multiplicity: số lần lặp codeword RM(1,7), vì RM(1,7) có độ dài 128 bit.

Ví dụ nhỏ

Với HQC-128:

$$k = 16, \quad n_1 = 46, \quad n_2 = 384.$$

Thông điệp m dài 16 byte. RS encode biến nó thành 46 byte. Mỗi byte đó được RM encode thành 384 bit. Do đó codeword nối tầng dài

$$46 \cdot 384 = 17664$$

bit.

3 Đầu vào và đầu ra của từng tầng decode

Đầu vào tổng thể của C .Decode là

$$w \in \mathbb{F}_2^{n_1 n_2}.$$

Có thể chia w thành n_1 khối:

$$w = (W_0, W_1, \dots, W_{n_1-1}),$$

trong đó

$$W_i \in \mathbb{F}_2^{n_2}.$$

Mỗi W_i là một duplicated RM codeword bị nhiễu. RM decoder biến mỗi khối W_i thành một byte:

$$\hat{a}_i = \text{RM.Decode}(W_i) \in \mathbb{F}_{2^8}.$$

Sau RM decode, ta thu được vector RS nhiễu:

$$\hat{a} = (\hat{a}_0, \dots, \hat{a}_{n_1-1}) \in \mathbb{F}_{2^8}^{n_1}.$$

RS decoder sửa lỗi symbol và trả về message:

$$m' \in \mathbb{F}_{2^8}^k.$$

4 Tầng Reed–Muller: duplicated RM(1,7)

4.1 RM(1,7) cơ bản

HQC dùng mã Reed–Muller bậc một:

$$\text{RM}(1, 7) = [128, 8, 64]_2.$$

Mỗi byte

$$a = (a_7, a_6, \dots, a_0) \in \mathbb{F}_2^8$$

được hiểu như hệ số của một hàm affine trên $\mathbb{F}_2^7$:

$$f_a(x_1, \dots, x_7) = a_7 + a_0 x_1 + a_1 x_2 + \dots + a_6 x_7.$$

Codeword RM là bảng giá trị của hàm này trên toàn bộ 128 điểm:

$$\text{RM.Encode}(a) = (f_a(x))_{x \in \mathbb{F}_2^7} \in \mathbb{F}_2^{128}.$$

Trong code C, ma trận sinh được encode bằng các mask bit:

bit message	pattern trong codeword 128 bit
a_0	1010 1010 ...
a_1	1100 1100 ...
a_2	1111 0000 ...
a_3	11111111 00000000 ...
a_4	16 bit 1, 16 bit 0, ...
a_5	32 bit 1, 32 bit 0, ...
a_6	64 bit 1, 64 bit 0
a_7	128 bit hằng

Đây là dạng nén của phép nhân:

$$c = a G_{\text{RM}},$$

với

$$G_{\text{RM}} \in \mathbb{F}_2^{8 \times 128}.$$

4.2 Duplicated RM

RM(1,7) chỉ dài 128 bit. HQC lặp nó μ lần:

$$\mu = \left\lceil \frac{n_2}{128} \right\rceil.$$

Với các tham số HQC hiện tại:

$$\mu = \begin{cases} 3, & \text{HQC-128,} \\ 5, & \text{HQC-192/HQC-256.} \end{cases}$$

Do đó:

$$W_i = \left(W_i^{(0)}, W_i^{(1)}, \dots, W_i^{(\mu-1)} \right), \quad W_i^{(j)} \in \mathbb{F}_2^{128}.$$

Nếu không có lỗi, tất cả các bản sao đều bằng cùng một RM codeword.

4.3 RM decode bước 1: expand and sum

Decoder cộng các bản sao theo từng vị trí bit:

$$S_t = \sum_{j=0}^{\mu-1} W_{i,t}^{(j)}, \quad t = 0, \dots, 127.$$

Do $W_{i,t}^{(j)} \in \{0, 1\}$, ta có

$$S_t \in \{0, 1, \dots, \mu\}.$$

Trong code:

```
expand_and_sum(&expanded, &codeArray[i * MULTIPLICITY]);
```

Ví dụ nhỏ

Giả sử $\mu = 3$ và chỉ nhìn 8 vị trí đầu:

$$W^{(0)} = 10110010, \quad W^{(1)} = 10100010, \quad W^{(2)} = 11110000.$$

Khi đó:

$$S = (3, 1, 3, 1, 0, 0, 2, 0).$$

Vector S cho biết ở mỗi vị trí có bao nhiêu bản sao mang bit 1.

4.4 RM decode bước 2: Fast Hadamard Transform

RM(1,7) được decode bằng biến đổi Hadamard nhanh. Ý tưởng chuẩn là đổi bit sang dấu:

$$b_t \in \{0, 1\} \mapsto (-1)^{b_t} \in \{+1, -1\}.$$

Với duplicated RM, sau khi cộng các bản sao, tương quan với từng codeword ứng viên có thể tính bằng Hadamard transform.

Code hiện tại không lưu trực tiếp ± 1 , mà lưu tổng bit 0/1. Vì vậy sau Hadamard có hiệu chỉnh:

$$T_0 \leftarrow T_0 - 64\mu.$$

Trong code:

```
hadamard(&expanded, &transform);  
transform[0] -= 64 * MULTIPLICITY;
```

Biến đổi Hadamard 128 điểm được thực hiện qua 7 pass. Ở mỗi pass, với cặp giá trị (x, y) :

$$(x, y) \mapsto (x + y, x - y).$$

Ví dụ nhỏ với Hadamard 4 điểm

Để minh họa, xét vector

$$s = (3, 1, 3, 1).$$

Hadamard 4 điểm:

$$\begin{aligned}(3, 1, 3, 1) &\mapsto (4, 4, 2, 2) \\ &\mapsto (8, 4, 0, 0).\end{aligned}$$

Với RM thật, kích thước là $128 = 2^7$, nên cần 7 tầng thay vì 2 tầng.

4.5 RM decode bước 3: tìm peak

Sau Hadamard, decoder chọn vị trí có trị tuyệt đối lớn nhất:

$$p = \arg \max_{0 \leq j < 128} |T_j|.$$

Vị trí p chứa 7 bit hệ số tuyến tính. Dấu của peak quyết định bit hằng a_7 . Code dùng quy tắc:

$$\hat{a} = \begin{cases} p + 128, & T_p > 0, \\ p, & T_p \leq 0. \end{cases}$$

Tức là byte output gồm:

$$\hat{a}_{0..6} = p, \quad \hat{a}_7 = \mathbf{1}_{T_p > 0}.$$

Trong code:

```
message_array[i] = find_peaks(&transform);
```

Ví dụ nhỏ

Giả sử Hadamard output có peak lớn nhất tại:

$$p = 37, \quad T_{37} = 52 > 0.$$

Khi đó byte decode là:

$$\hat{a} = 37 + 128 = 165.$$

Nếu $T_{37} = -52$, byte decode là:

$$\hat{a} = 37.$$

5 Đầu ra RM là đầu vào RS

Sau khi RM decode toàn bộ n_1 khối, ta có:

$$\hat{a} = (\hat{a}_0, \dots, \hat{a}_{n_1-1}) \in \mathbb{F}_{2^8}^{n_1}.$$

Đây là một Reed–Solomon codeword bị lỗi symbol:

$$\hat{a} = c_{\text{RS}} + e_{\text{RS}}.$$

Lỗi bit còn sót lại sau RM thường biểu hiện thành lỗi byte/symbol đối với RS.

Ví dụ nhỏ

Giả sử RS codeword đúng là:

$$c_{\text{RS}} = (12, 55, 81, 9, 200, 7).$$

RM decode trả về:

$$\hat{a} = (12, 55, 44, 9, 200, 7).$$

Khi đó có một lỗi symbol tại vị trí 2:

$$e_{\text{RS}} = (0, 0, 81 + 44, 0, 0, 0),$$

trong đó phép cộng là XOR trong $\mathbb{F}_{2^8}$.

6 Tầng Reed–Solomon trên $\mathbb{F}_{2^8}$

6.1 Trường hữu hạn

RS của HQC làm việc trên:

$$\mathbb{F}_{2^8} = \mathbb{F}_2[z]/p(z),$$

với đa thức bất khả quy:

$$p(z) = z^8 + z^4 + z^3 + z^2 + 1.$$

Trong code, đa thức này được biểu diễn bởi:

$$\text{PARAM_GF_POLY} = 0x11D.$$

Mỗi symbol RS là một byte, tức một phần tử của $\mathbb{F}_{2^8}$.

Ví dụ nhỏ

Byte `0x53` biểu diễn đa thức:

$$z^6 + z^4 + z + 1$$

vì

$$0x53 = 01010011_2.$$

Khi nhân hai byte trong RS decoder, ta nhân hai đa thức trên $\mathbb{F}_2$, rồi lấy modulo $p(z)$.

6.2 Generator polynomial của RS

RS code trong HQC là shortened Reed–Solomon code. Generator polynomial có dạng:

$$g(x) = \prod_{i=1}^{2\delta} (x - \alpha^i),$$

trong đó α là phần tử nguyên thủy của $\mathbb{F}_{2^8}$, và δ là khả năng sửa lỗi.

Trong code, các hệ số $g(x)$ được precompute:

$$g(x) = g_0 + g_1x + \cdots + g_{2\delta}x^{2\delta}.$$

Ví dụ HQC-128 có $\delta = 15$, nên:

$$\deg g = 30, \quad \text{số hệ số} = 31.$$

Mảng `RS_POLY_COEFS` trong `parameters.h` chứa các hệ số này.

6.3 RS decode tổng quát

Input của RS decoder:

$$\hat{a} = (\hat{a}_0, \dots, \hat{a}_{n_1-1}) \in \mathbb{F}_{2^8}^{n_1}.$$

Output:

$$m' = (m'_0, \dots, m'_{k-1}) \in \mathbb{F}_{2^8}^k.$$

Trong code, RS decode có sáu bước:

1. Tính 2δ syndrome.
2. Tính error locator polynomial $\sigma(x)$ bằng Berlekamp.
3. Tìm nghiệm của $\sigma(x)$ bằng additive FFT, suy ra vị trí lỗi.
4. Tính error evaluator polynomial $z(x)$.
5. Tính giá trị lỗi tại từng vị trí.
6. Sửa codeword và lấy lại k byte message.

7 RS bước 1: tính syndromes

Với received word:

$$r(x) = r_0 + r_1x + \dots + r_{n_1-1}x^{n_1-1},$$

syndrome được tính tại các nghiệm của $g(x)$:

$$S_i = r(\alpha^{i+1}), \quad i = 0, \dots, 2\delta - 1.$$

Trong code:

```
compute_syndromes(syndromes, cdw_bytes);
```

với công thức triển khai:

$$S_i = r_0 + \sum_{j=1}^{n_1-1} r_j \alpha^{(i+1)j}.$$

Ví dụ nhỏ

Giả sử một RS code nhỏ có $2\delta = 4$. Ta tính:

$$S_0 = r(\alpha), \quad S_1 = r(\alpha^2), \quad S_2 = r(\alpha^3), \quad S_3 = r(\alpha^4).$$

Nếu tất cả syndrome bằng 0:

$$S_0 = S_1 = S_2 = S_3 = 0,$$

thì $r(x)$ là codeword hợp lệ. Nếu khác 0, decoder tiếp tục tìm vị trí và giá trị lỗi.

8 RS bước 2: error locator polynomial

Giả sử có $\nu \leq \delta$ lỗi tại các vị trí $j_1, \dots, j_\nu$. Đặt error locator numbers:

$$\beta_\ell = \alpha^{j_\ell}.$$

Error locator polynomial là:

$$\sigma(x) = \prod_{\ell=1}^{\nu} (1 - \beta_\ell x).$$

Trong đặc số 2, dấu trừ giống dấu cộng:

$$1 - \beta_\ell x = 1 + \beta_\ell x.$$

Khai triển:

$$\sigma(x) = 1 + \sigma_1 x + \dots + \sigma_\nu x^\nu.$$

Code tính $\sigma(x)$ bằng biến thể constant-time của thuật toán Berlekamp:

```
deg = compute_elp(sigma, syndromes);
```

Ví dụ nhỏ

Nếu có hai lỗi với locator numbers β_1, β_2 , thì:

$$\sigma(x) = (1 + \beta_1 x)(1 + \beta_2 x) = 1 + (\beta_1 + \beta_2)x + \beta_1 \beta_2 x^2.$$

Nghiệm của $\sigma(x)$ cho phép suy ra vị trí lỗi.

9 RS bước 3: tìm vị trí lỗi bằng nghiệm của $\sigma(x)$

Decoder cần tìm các vị trí j sao cho có lỗi tại symbol r_j . Theo quy ước trong implementation, code dùng FFT để đánh giá $\sigma(x)$ trên các điểm của $\mathbb{F}_{2^s}$, rồi khôi phục error polynomial:

$$E_j = \begin{cases} 1, & \text{nếu vị trí } j \text{ có lỗi,} \\ 0, & \text{ngược lại.} \end{cases}$$

Trong code:

```
compute_roots(error, sigma);
```

Hàm này gọi:

```
fft(w, sigma, PARAM_DELTA + 1);  
fft_retrieve_error_poly(error, w);
```

Ví dụ nhỏ

Giả sử $n_1 = 6$ và FFT cho biết:

$$E = (0, 0, 1, 0, 0, 1).$$

Khi đó decoder biết có lỗi ở vị trí:

$$j = 2, \quad j = 5.$$

10 RS bước 4: error evaluator polynomial $z(x)$

Sau khi có $\sigma(x)$, decoder tính polynomial:

$$z(x) = 1 + z_1x + \cdots + z_\delta x^\delta.$$

Trong code, $z(x)$ được tính từ $\sigma(x)$ và syndromes:

```
compute_z_poly(z, sigma, deg, syndromes);
```

Về mặt ý tưởng, $z(x)$ đóng vai trò tương tự error evaluator polynomial trong Forney formula: nó kết hợp thông tin syndrome và locator polynomial để tính độ lớn lỗi tại từng vị trí.

Ví dụ nhỏ

Nếu:

$$\sigma(x) = 1 + \sigma_1x + \sigma_2x^2,$$

và syndrome đầu là $S_0, S_1, \dots$, thì các hệ số thấp của $z(x)$ được tạo bằng các tổ hợp kiểu:

$$z_1 = \sigma_1 + S_0,$$

$$z_2 = \sigma_2 + S_1 + \sigma_1 S_0,$$

trong $\mathbb{F}_{2^8}$. Code dùng mask để chỉ giữ các hệ số phù hợp với bậc thật của σ .

11 RS bước 5: tính giá trị lỗi

Với mỗi vị trí lỗi j_i , code tính:

$$\beta_{j_i} = \alpha^{j_i}.$$

Sau đó tính error value e_{j_i} theo công thức dạng Forney. Trong code:

$$e_{j_i} = \frac{1 + \sum_{j=1}^{\delta} z_j \beta_{j_i}^{-j}}{\prod_{k \neq i} (1 + \beta_{j_i}^{-1} \beta_{j_k})}.$$

Tất cả phép toán đều trong $\mathbb{F}_{2^8}$.

Trong code:

```
compute_error_values(error_values, z, error);
```

Ví dụ nhỏ

Giả sử có hai lỗi tại $j_1 = 2, j_2 = 5$. Khi đó:

$$\beta_{j_1} = \alpha^2, \quad \beta_{j_2} = \alpha^5.$$

Với lỗi thứ nhất:

$$e_{j_1} = \frac{1 + z_1 \beta_{j_1}^{-1} + z_2 \beta_{j_1}^{-2} + \cdots}{1 + \beta_{j_1}^{-1} \beta_{j_2}}.$$

Kết quả là một byte trong $\mathbb{F}_{2^8}$, ví dụ $e_{j_1} = 0x7A$.

12 RS bước 6: sửa lỗi và lấy message

Sau khi có vector error values:

$$e = (e_0, \dots, e_{n_1-1}),$$

decoder sửa:

$$c_{\text{RS}} = r + e.$$

Vì trên đặc số 2, cộng là XOR:

$$c_{\text{RS},j} = r_j \oplus e_j.$$

Trong code:

```
correct_errors(cdw_bytes, error_values);
```

RS encode trong HQC là systematic. Message nằm ở phần cuối codeword sau parity:

$$m' = (c_{\text{RS},g-1}, \dots, c_{\text{RS},g-1+k-1}),$$

trong đó:

$$g = \text{PARAM_G} = 2\delta + 1.$$

Code lấy message bằng:

```
memcpy(msg, cdw_bytes + (PARAM_G - 1), PARAM_K);
```

Ví dụ nhỏ

Giả sử $n_1 = 6$, $k = 3$, parity dài 3, codeword systematic:

$$c_{\text{RS}} = (p_0, p_1, p_2, m_0, m_1, m_2).$$

Sau khi sửa lỗi xong, decoder trả về:

$$m' = (m_0, m_1, m_2).$$

13 Tóm tắt toàn bộ flow bằng công thức

Decrypt PKE:

$$\begin{aligned} y &\leftarrow \text{SampleSecret}(dk), \\ w &= v + \text{Truncate}(uy). \end{aligned}$$

Chia w :

$$w = (W_0, \dots, W_{n_1-1}), \quad W_i \in \mathbb{F}_2^{n_2}.$$

RM decode từng block:

$$\hat{a}_i = \text{RM.Decode}(W_i), \quad i = 0, \dots, n_1 - 1.$$

Ghép thành RS received word:

$$\hat{a} = (\hat{a}_0, \dots, \hat{a}_{n_1-1}).$$

RS decode:

$$\begin{aligned} S_i &= \hat{a}(\alpha^{i+1}), \quad i = 0, \dots, 2\delta - 1, \\ \sigma(x) &\leftarrow \text{Berlekamp}(S_0, \dots, S_{2\delta-1}), \\ E &\leftarrow \text{RootsByFFT}(\sigma), \\ z(x) &\leftarrow \text{Evaluator}(\sigma, S), \\ e &\leftarrow \text{ForneyLikeValues}(z, E), \\ c_{\text{RS}} &= \hat{a} + e, \\ m' &= \text{systematic message part of } c_{\text{RS}}. \end{aligned}$$

14 Mapping sang các file code

File/hàm	Vai trò
src/ref/hqc.c	Tính $w = v + \text{Truncate}(uy)$, rồi gọi <code>code_decode</code> .
src/common/code.c	Nối tầng decoder: RM decode trước, RS decode sau.
src/ref/reed_muller.c	Decode duplicated RM(1,7): expand-and-sum, Hadamard, find peak.
src/ref/reed_solomon.c	Decode RS: syndrome, Berlekamp, FFT roots, $z(x)$, error values, correction.
src/ref/gf.c	Toán $\mathbb{F}_{2^8}$: nhân, bình phương, nghịch đảo modulo $0x11D$.
src/ref/hqc-*/parameters.h	Tham số $n, n_1, n_2, k, \delta, g$, generator polynomial RS.

Table 2: Các điểm bám chính trong implementation.

15 Pseudocode decode HQC RMRS

```

Input:
  ciphertext c = (u, v)
  secret key dk that regenerates y

Output:
  decoded message m'

1. y <- parse/regenerate secret vector from dk
2. t <- Truncate(u * y) // polynomial multiplication in R
3. w <- v XOR t // because subtraction over F_2 is XOR

4. // Reed-Muller layer
5. for i = 0 to n1 - 1:
6. W_i <- w[i*n2 : (i+1)*n2 - 1]
7. Split W_i into multiplicity blocks W_i^(0), ..., W_i^(mu-1)
8. S <- expand_and_sum(W_i^(0), ..., W_i^(mu-1))
9. T <- Hadamard128(S)
10. T[0] <- T[0] - 64*mu
11. p <- argmax_j |T[j]|
12. a_hat[i] <- p + 128*(T[p] > 0)

13. // Reed-Solomon layer
14. S_syn <- compute_syndromes(a_hat)
15. sigma <- Berlekamp(S_syn)
16. error_locations <- roots_by_additive_fft(sigma)
17. z <- compute_z_poly(sigma, S_syn)
18. error_values <- compute_error_values(z, error_locations)
19. c_rs <- a_hat XOR error_values
20. m' <- systematic message bytes of c_rs

```

16 Ghi chú quan trọng

- RM sửa lỗi ở mức bit trong từng block n_2 bit. Output của RM là các symbol byte.
- RS sửa lỗi ở mức symbol byte. Một byte sai sau RM được xem là một symbol error.

- Decode nối tầng phải đi theo thứ tự RM rồi RS, vì encode đi theo thứ tự RS rồi RM.
- Tầng RS có thể sửa tối đa δ symbol lỗi, với δ phụ thuộc variant.
- Các phép trong RM là trên $\mathbb{F}_2$, còn các phép trong RS là trên $\mathbb{F}_{2^8}$.
- Implementation dùng constant-time style ở các phần nhảy cảm, đặc biệt Berlekamp và xử lý error values.